Conditional statements

if :

elif :

else :

finally :

Loops

① for ( )

included ↓ excluded ↓ jump ↓

for i in range (start, stop, step):

→ eg for i in range (1, 10, 2):
print (i)

→ 1 3 5 7 9

→ eg. for i in range (10, 1, -2):
print (i)

→ 10
8
6
4
2

② while ()

~~while 5>26~~
~~print~~

```
a=2
while a < 3:
    print(a)
    a+=1
```

=> 2

---

④ break → breaks out of the innermost loop.

④ continue → skips the current iteration and continues with the next.

④ pass → null statement, does nothing.

Data Structures

① Lists → a = list() or a = [ ]

List methods

① l.remove() → removes the first occurrence of the element.

② l.pop() → ~~returns~~ removes & returns the last element in the list

③ l.index() → returns the index of the given element.

④ l.count() → ~~count~~ returns the count of a specific element.

⑤ l.insert(2, 'x') → inserts x at 2 index in the list.

⑥ l.sort() → sorts the list in ascending order.

⑦ l.reverse() → reverses the list.

⑧ l.clear() → clears the list.

~~⑨ enumerate( )~~ → used to find index of each element.
→ runs 2 identifiers in a single range / list
→ to iterate over indices & values

eg: for i, j in enumerate([1,2,3,4]):
print(i, j)

⇒

index	value
0	1
1	2
2	3
3	4

Enumerate ()

→ To iterate over a list along with the index.

eg: for i, j in enumerate([5, 10, 15, 20]):
    print (i, j)

⇒

0	5
1	10
2	15
3	20

indexes values

List comprehension

eg — even_num = [num for num in range(10) if num % 2 == 0]
print (even_num)

⇒ [0, 2, 4, 8]

Note → Basic syntax	[expression for item in iterable]
with conditional logic	[expression for item in iterable if condition]
nested	[exp for item1 in iterable1 for item2 in iterable2]

len ( ) → returns length of the argument

sorted ( )	list. sort ( )
→ works on any iterable	→ only for list
→ returns a new sorted list	→ returns None
→ no effect on original	→ sorts the original list in place

eg → reverse sorting
x = sorted (list, reverse = True)

[. . .]
[. . .]

i \ j	1	2	3
1	1	1	1
2	0	1	0
3	2	0	1

~~x for x in y~~

~~x for x in y for y in z~~

for i in range (len (m)):
for j in range (len(m)) ~~x [for y in z] for x in y~~
[ ] [ ]
1 1 0 0 1 0 1 1 0 1 0 0

~~for [x for y in z for x in y]~~
not english

a[i][j] for

(0,0)	(0,1)	(0,2)
(1,0)	(1,1)	(1,2)
(2,0)	(2,1)	(2,2)

```
for i in (0,2)
    for j in (0,2)
        print a(i,j)
```

~~(0,0)~~ (0,0) (1,0) (2,0)

(0,1) (1,1) (2,1)

List rotation (n times)

```
-> def rr(lis, n)
      return lis[n:] + lis[:n]
```

easiest way

Missing number (To n integers)

lis = [1, 2, 4, 5, 8]

① → Take XOR of n ints as well as whole list.

② → Sum of n numbers − sum of list.

(*) Mutable objects can only be referenced using a '=' sign & not copied.

eg → list, dict, matrix

(*) Immutable objects have a copy made when = is used.

Note – Reference bugs only matter if the object can be modified in place.

(*) → id( ) → unique identifier while the program is running
→ represents where the object lives in memory.

→ helps to check if 2 variables point to the same object.

(*) Zip → zip( ) to join/iterate through 2 lists of varied length.